KiroClops — Technical Report

OpenCV AI Competition 2026, powered by AWS. Solo entry: one developer designed, built and evaluated the project, using AI coding and review agents as disclosed in §6.1. Version of 23 October 2026. Every number below is taken from a frozen evidence folder named in the text; nothing was re-run to make it look better, and failed attempts are reported as recorded.

Summary

KiroClops lets a developer show the behaviour they want as a short video and have a coding agent (Kiro CLI) implement it in their real application, while OpenCV 5 measures whether the rendered result actually matches. The measurement harness tracks objects in reference recordings and in headless re-recordings of every candidate the agent submits, extracts motion events and connector attachments, and decides pass/fail against pre-registered, hash-bound thresholds. Held-out recordings are revealed only after the agent seals a candidate; every run leaves an evidence chain (sources, frames, measurements, native tool-call audit) that a reviewer who never saw the production coordinates can replay.

Verified state at submission:

Domain	Result	Evidence
Game (Godot, jump and ground motion)	**Accepted**: behaviour regression 48/48 and held-out validation 30/30; measurement, preserved functions, independent raw review, source review and native audit all pass	closure `v4g01-closure-02` (`evidence/closures/`)
SVG diagram (node add/remove, drag, connector attachment)	**Accepted**: 28 window results pass, 112 raw records (244 sheets, 480 points) independently reviewed, 32/32 blind focus-ring marks within 2 px	closure `v5s01c-closure-01` (`evidence/closures/`)
Web panel (spring animation with interruptions)	**Not accepted**: best run 55/56 held-out comparisons with one drag-release regression; four earlier attempts failed on behaviour or on recording stalls	`evidence/panel-v6p01/heldout-outcome.json`
AWS	Live analysis service on a Graviton4 worker running the COOL build of OpenCV (5.1.0-dev); 17/17 recordings equivalent to the local 5.0.0 measurement	`evidence/aws/`, §5

What did not work. These are part of the result, not footnotes.

item	what happened	what we did
Web panel domain	Not accepted in four attempts: V4 27/36 regression cases; V5 no eligible candidate; V5b a capture stall during finalization; V6 55/56 held-out cases with one drag-release regression	Every run is reported as failed. The next run adds drag-release cases to calibration under a new identifier
COOL speed	No measurable speed-up on this workload (paired ratio 1.003, 95% interval 0.991–1.010)	Reported as measured; §5.2 shows where the time goes
Learned features on tiny glyphs	ALIKED and LightGlue locate only 45% of the small, low-texture handle glyph	Used as a cross-check, never as a gate
Native audit	Kiro CLI updated itself to 2.31, outside the certified 2.27.x rules	The V6 run is reported as native-uncertified
`inspect_window`	Placed windows on the recording's first input and plotted absolute positions	Fixed after the freeze; never used in a recorded run (§6.4)

Reading guide for judges.

judging criterion	where to look
Technical execution (30%)	§3: OpenCV 5 tracking, OpenCV 5-specific features, measured accuracy. §6.4: tests

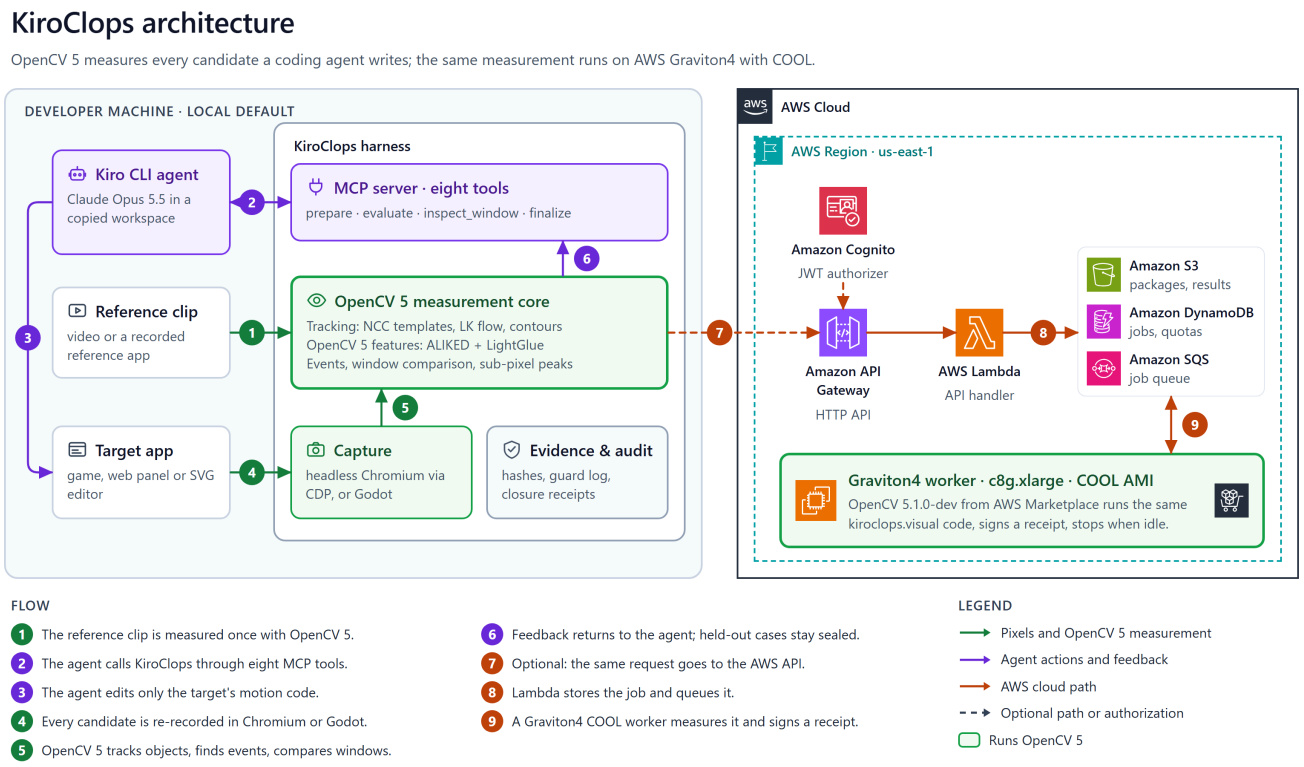
judging criterion	where to look
Innovation (20%)	§1 and §4: a referee for coding agents, with held-out cases sealed until the agent commits
Real-world impact (20%)	§1: users. §6.2: results. §4.2 and `AGENTIC_VISION.md` §3: the sealed candidate is 7.5 times closer to the reference than the app the agent started from, and all 17 candidates in 13 logged sessions beat the unchanged app
User experience (10%)	The demo video, and `JUDGE_ACCESS.md`: one command against the live service
Documentation and presentation (10%)	This report, `README.md`, the diagrams, `evidence/README.md`
Cloud delivery, reproducibility and responsible operation (10%)	§5: AWS, COOL, receipts, judge access. §7: limitations. §8: reproducibility

1. Problem and users

Motion is specified by example far more often than by numbers: "open like this card", "jump like that character", "re-layout like this diagram". Today a developer either tunes constants by eye or describes the motion to an AI coding assistant and trusts the result. Neither step is measured. KiroClops closes that loop for three kinds of 2D rendered motion: game characters, web UI animations and structural canvas/SVG interfaces.

Intended users are game, front-end and interactive-tool developers who already use an AI coding agent. They supply a reference clip (a video, or a recording of a reference implementation) and a plain-language request; KiroClops supplies the measurement, the calibration feedback the agent works against, and the final verdict. We make no claim about external users: no usability study was run, and the "real benefit" evidence is limited to the measured improvements in the sections below.

2. Architecture



- **Measurement core** (`kiroclops/visual`, about 2,700 lines) depends only on OpenCV 5 and NumPy and imports nothing from the other packages. Its output, an `ObservationSet` per recording, is immutable evidence identified by hash.

- **Capture** (`kiroclops/web_local`, `adapters/`) records web targets through the Chrome DevTools Protocol with per-frame host and page clock brackets, and game targets through a Godot adapter at a fixed physics rate. Recordings that miss timing limits are rejected, never patched.
- **Agent loop** (`kiroclops/local_engine`) runs a Kiro CLI session against a copied target with eight MCP tools, a pre-tool-use guard, bounded text feedback, and a post-run audit of Kiro's own logs. Section 4 describes it.
- **Evidence chain**: pre-registered recipes and thresholds; source epochs frozen as byte-identical trees with SHA-256 manifests (tree hashes in `evidence/source-epochs.json`); closure receipts that bind measurement, behaviour, preserved functions, independent raw review, source review and native origin into one validated result.
- **AWS service** (`services/analysis_api`, `kiroclops/cloud`) runs the same measurement code on a Graviton4 worker behind an authenticated HTTP API, with receipts and a cross-engine equivalence check. Section 5.

Both diagrams are rendered from `diagrams/*.html` by `diagrams/build_diagrams.py`, which measures every label, box and arrow and refuses to export a diagram with overflowing text, overlaps or crossing edges; vector PDFs sit next to the PNGs. AWS service icons are the official AWS Architecture Icons (Q3 2026 package).

3. OpenCV 5 implementation

3.1 Tracking and events

Each selected object is tracked by one of three methods:

- **Fixed-template NCC**: `cv2.matchTemplate` (`TM_CCOEFF_NORMED`) over a bank of mirror and quarter-turn variants of the selected patch, with a colour-mean gate (`cv2.integral`) that preserves identity under channel offsets and a distinct-location margin gate that rejects ambiguous matches. The search radius scales with the observed frame interval, never with a prediction; invalid frames carry no coordinates.
- **Lucas-Kanade flow**: `cv2.goodFeaturesToTrack`, `cv2.calcOpticalFlowPyrLK` with a forward-backward check, and a RANSAC similarity from `cv2.estimateAffinePartial2D`.
- **Contour geometry**: `cv2.Canny`, `cv2.findContours`, `cv2.boundingRect` with a rectangularity gate, used for separately measured boundaries.

Connectors between SVG nodes are straight segments found by Canny edges, a corridor mask, `cv2.dilate`, `cv2.connectedComponentsWithStats` and `cv2.fitLine`; coloured polylines use a colour mask and thinning. A ring-aware rule (V5) removes the achromatic focus ring Chromium paints around a focused node from the stroke mask and flags an endpoint as occluded when the ring covers it; the attachment decision then treats an occluded endpoint as attached. In a blind qualification, 32 of 32 production endpoints were within 2 px of independent marks and 32 of 32 attachment decisions agreed (`evidence/svg-focus-ring-blind-qualification.json`).

Events are extracted from the tracks: rest-band departure, confirmed rest, reversal, overshoot, oscillation, boundary crossing and overlap relations, and connector attach/detach, each with a time bracket and an uncertainty. Comparison aligns reference and candidate on their input origins and evaluates windows of position RMSE, endpoint and size error (as fractions of a physical scale), required event order and dynamics such as settling time.

3.2 What is specific to OpenCV 5

We diffed the Python symbol tables of `opencv-python` 4.14.0.94 and 5.0.0.93: 194 symbols exist only in 5.0 and 166 were removed (`evidence/opencv5-symbol-diff`). KiroClops uses the following 5.0-only capabilities, and refuses to measure on a 4.x runtime (`kiroclops/visual/opencv5.py`); every `ObservationSet` records the OpenCV build it was produced with.

- **Learned local features** from the new `features` module: `cv2.ALIKED` keypoints (sub-pixel, 128-D descriptors) matched by `cv2.LightGlueMatcher`, with the model files OpenCV's own test suite pins (`opencv_extra download_models.py`, SHA-1 verified by `scripts/fetch_opencv5_models.py`). They provide a scale- and rotation-tolerant locator (`method: features`) and an independent cross-check of the template tracker (`cross_check: features_v1`).
- With the 5.0.0 wheel on our Windows machine, LightGlue aborts the process when a later call has more keypoints than the first one, and its results depend on earlier calls; when the network finds no match, the call fails with an integer division by zero. We documented both and isolated them: a warm-up at the ALIKED limit (1,000 × 1,000

keypoints) and a helper process that reports an empty result as `matcher_engine_failure`. Validated on 1,617 keypoint pairs in two random orders with identical results and no restarts (`evidence/lightglue-sweep`). After the freeze we ran the same inputs on Linux x86-64 and AArch64 with the 5.0.0 wheel, a 5.1.0-dev build and the current 5.x branch built from source (`evidence/lightglue-platforms`). The abort and the order dependence occur only on Windows. The division by zero, in the DNN engine's `GatherND` layer, kills the process on Linux x86-64, silently returns no matches on AArch64 and is still present in 5.x. We reported both upstream (opencv/opencv#30501 and opencv/opencv#30502).

- `cv2.estimateAffinePartial2D` in 5.0 accepts RANSAC/LMEDS only (USAC methods raise), which the code and tests reflect.

Everything else in the pipeline (`matchTemplate`, `calcOpticalFlowPyrLK`, `estimateAffinePartial2D`, `Canny`, `fitLine`, `connectedComponentsWithStats`, `phaseCorrelate`) is classical OpenCV used on purpose: the accepted results depend on exact, replayable behaviour, not on a learned model.

3.3 Measured accuracy

Sub-pixel refinement. Integer NCC quantises positions to half a pixel. V6 adds an opt-in refinement that enlarges the template and the integer-peak window with the same bicubic kernel (`cv2.resize`) and scores them with the same normalised cross-correlation. On synthetic bilinear shifts of 0.1–0.5 px with noise σ up to 5 and a gamma change (`evidence/subpixel-benchmark`, 30 trials per condition):

method	RMSE px (text card)	RMSE px (smooth texture)	max px
integer peak	0.373	0.373	0.636
three-point parabola (`parabolic_v1`)	0.075–0.076	0.054–0.056	0.138
symmetric upsampling (`upsampled_v1`)	0.050–0.052	0.038–0.040	0.097

Against independent marks on real recordings (`evidence/marks-error`): 117 panel points and 216 SVG points were located in raw pixels by a separate extraction path that never sees production coordinates, and the sheets were then checked visually by AI reviewer agents. On the panel, the template tracker's scatter around its systematic offset falls from 0.41 px (integer) to 0.24 px (`upsampled_v1`); per glyph, from 0.28–0.34 px to 0.04–0.05 px. The accepted measurement (pixel-anchor centroid) has 0.01–0.02 px scatter per glyph plus a constant ±1 px per-glyph definition offset that the comparison cancels. The SVG reviewer positions equal the accepted measurement exactly, so they confirm reproducibility rather than independence.

Learned-feature cross-check on real recordings (`evidence/feature-cross-check`): on five SVG calibration recordings ALIKED+LightGlue located 3,543 of 3,584 object-frames (98.9%) with a median distance of 0.35–0.91 px from the template tracker and scale ≈ 1.00. On sixteen panel recordings it located 78.7% of object-frames: 96.4% of the bookmark and 94.6% of the star glyph at 0.7–1.0 px median (mostly the per-glyph definition offset above), but only 45% of the small, low-texture handle glyph, where LightGlue returns no matches. We report this as the honest limit of learned features on tiny UI glyphs; the cross-check is diagnostic and never gates a result.

Equivalence of the V6 tracker with the accepted epochs: replaying 48 frozen panel and 5 SVG recordings with V6 defaults reproduces every stored observation bit for bit (`evidence/v6-tracker-equivalence`), so the accepted closures remain valid under the current code.

4. The agent loop

4.1 Tools and control flow

The launcher writes a Kiro agent definition, a pre-tool-use hook and the task prompt into a fresh copy of the target, hashes every control file and pins them to the session. The agent then has eight MCP tools:

tool	purpose
`inspect_session`	domain, editable scope, remaining candidates and wall-clock budget, measured evaluation time

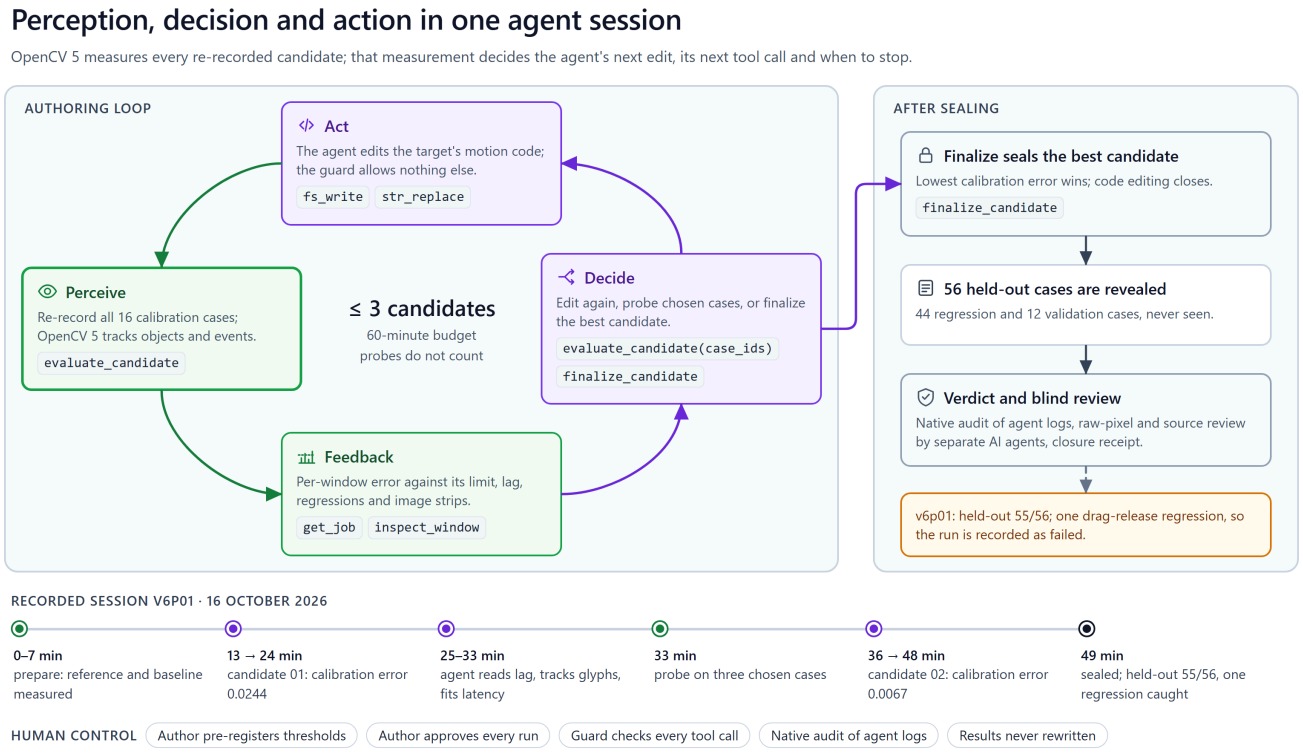
tool	purpose
`prepare_experiment`	record and measure the reference and the unchanged baseline for every calibration case
`evaluate_candidate(case_ids?)`	snapshot the edited target, re-record every calibration case, compare; with `case_ids`, a faster partial probe that never counts as a candidate and cannot be finalized
`inspect_window(job_id, case_id, object_id, window_id?)`	write reference/candidate/difference PNG strips and an error-over-time plot into the workspace, aligned and in the coordinate space of the scored window (post-freeze fix, §6.4)
`finalize_candidate`	seal the lowest-error eligible candidate; held-out cases are revealed only now
`get_job`, `cancel_job`, `build_report`	job progress and results, cancellation, the evidence report

Budgets are frozen per session: three full candidates, a 60-minute authoring clock that starts with the task, and a separate validation clock after sealing. Feedback is bounded text under Kiro's 30,000-character offload limit: per-window errors against their limits, whether a window got worse than the unchanged baseline, signed offsets and a lag estimate, and the path of every candidate frame package. Everything larger stays in the workspace as JSON.

4.2 Why this qualifies as a perception-decision-action loop

OpenCV 5 output changes what the agent does next. In the V6 panel run (session `v6p01`, 230 audited tool calls: `get_job` 126, `execute_pwsh` 61, `fs_write` 18, `str_replace` 7, `read_file` 6, `inspect_session` 4, `evaluate_candidate` 3, `prepare_experiment` 1, `cancel_job` 1, `finalize_candidate` 1) the agent submitted candidate 01 (calibration error 0.0244), read the per-window measurement feedback, edited the motion code and submitted candidate 02 (0.0067, 3.6× lower), then chose it for finalization. Scored with the same formula, the unchanged app the agent started from has a calibration error of 0.0502, so the sealed candidate is 7.5 times closer to the reference. On the 56 held-out cases it is 4.1 times closer (0.0121 against 0.0500) and better on 53 of them. This is a post-hoc recomputation from the recorded per-window errors (`evidence/v6-agent-baseline/result.json`). The agent's own helper scripts, preserved in the workspace, show how it used the visual evidence: it printed every window's error, limit and the lag KiroClops measured (about −10 ms on candidate 01's worst windows), tracked the panel glyphs in candidate 01's exported frames, fitted the per-input latency of its motion model to those tracks, and then wrote an input-clock model and re-tuned the spring; candidate 02's measured lag is 0. It also started one partial probe on three chosen cases, one from each recipe family added in V4/V5, and cancelled it to apply a last fix. It did not use `inspect_window` in that run; we report tool usage as recorded. The trace is `kiro/tool-audit.jsonl` plus the job results in the session (`AGENTIC_VISION.md`).

The other logged sessions show the same pattern (`evidence/agent-sessions/REPORT.md`, post-hoc and read-only, every recorded score reproduced). Over all 13 Kiro sessions from 1 to 16 October (7 web panel, 6 game, 1,135 audited tool calls), all 17 measured candidates were closer to the reference than the unchanged app on calibration; in 5 of the 6 sessions with a second candidate, written after the feedback, the error fell (by 5–73%). Of the 7 sessions that reached sealed held-out scoring, all 7 sealed candidates beat the unchanged app there too (on 308 of 326 cases), but only one passed every held-out case. These are single runs under recipes that changed between versions, not a controlled experiment.



4.3 Control, isolation and audit

- **Guard hook**: reads stay inside the target; writes are limited to the motion source files and a scratch folder; session folders, parent traversal, control files and unknown path keys are denied; writes and shell close after sealing. Every decision is recorded before the call proceeds. This is a cooperative experiment boundary, not an OS sandbox, and the report says so.
- **Native audit**: after the turn, Kiro's own transport logs are replayed against pinned envelope rules and matched one-to-one with the hook's decisions. The rules are certified for kiro-cli 2.27.x; the CLI updated itself to 2.31.0 before the last panel run, so that run is reported as native-uncertified.
- **Human control**: the author pre-registered every protocol, threshold and recipe, and approved each run, each temporary removal of the global Kiro power and each post-hoc label; results are never rewritten; a failed run is kept and a rerun gets a new identifier; both accepted-domain closures were validated by replay.

5. AWS deployment and COOL

5.1 Service

`services/analysis_api` is a CloudFormation stack: Cognito (human and agent clients), an HTTP API with a Lambda handler, S3 for packages and results, DynamoDB for jobs and quotas (compare-and-swap writes), SQS, and one c8g.xlarge Graviton4 worker that runs each job in a `systemd-run` sandbox (private network, 2 GB memory, 120 s limit, unprivileged user). The worker executes the same `kiroclops.visual` code as the local engine and returns a receipt that binds package hash, request hash, engine identity and measurement hash. Quotas: one active job per tenant, 1,800 compute-seconds per day, 2 GiB storage; invitations are limited to five accounts. A controller Lambda stops the worker after five idle minutes and wakes it on new jobs.

5.2 COOL on the worker

The live deployment boots the worker from the AWS Marketplace "Cloud Optimized OpenCV for AWS Graviton4" AMI (`ami-033e481a24f94c8cb`, OpenCV 5.1.0-dev, NumPy 2.5.1 under `/opt/cool`). The worker's virtual environment imports COOL's `cv2` through a `.pth` entry and ldconfig, and boot fails unless `cv2` resolves under `/opt/cool/`. COOL's `cv2`

import contacts the EC2 instance metadata service, which the sandbox's private network blocked; a systemd drop-in allows that one address only (the security cost is that the sandbox can then read the instance role's credentials; §7 lists it). Each delivery receipt names the engine, so the COOL path is proven per job rather than claimed.

Equivalence. The same 17 recordings (16 V6 panel calibration references and one Kiro candidate) measured locally (Windows x86, OpenCV 5.0.0) and on the worker: standard wheel 17/17 equivalent under the unchanged policy (max numeric difference 1.70e-6); COOL 17/17 after excluding the `opencv_runtime` build record that V6 adds to every result, max difference 1.43e-6, 96.4% of 285,031 values bit-identical (`evidence/aws/cool-equivalence`, labelled post-hoc because the policy predates that field).

Protocol. Image `ami-033e481a24f94c8cb` (image name `Graviton5-COOL-v3-prod-k6u24vxijyzvg`) of the AWS Marketplace product "Cloud Optimized OpenCV for AWS Graviton4", reporting OpenCV 5.1.0-dev and NumPy 2.5.1 from `/opt/cool`. The standard arm uses the `opencv-python` 5.0.0.93 wheel with NumPy 2.2.6. Both run on c8g.xlarge (four Graviton4 vCPUs) with four OpenCV threads. In the speed test below the two engines ran on separate instances (a fresh standard worker and the live COOL worker). The COOL hourly price includes the $0.01/h Marketplace fee. x86 and Arm coexist: the client and the local engine run on x86 Windows, the service worker on Arm, and the equivalence check above shows that both produce the same tracks and events.

Speed and cost (`evidence/aws/speed`, 21 October). The same 17 recordings were submitted three times each to a fresh standard-wheel worker (OpenCV 5.0.0) and to the live COOL worker (OpenCV 5.1.0-dev), both c8g.xlarge with four threads, with a probe tag in each request so the service's exact-reuse cache never answered; the same recordings were measured three times locally (Windows x86, OpenCV 5.0.0). Worker time is the receipt's `worker_wall_seconds`: from claiming the job to the finished measurement, including the S3 download and integrity check, the sandbox start and the complete analysis.

engine	OpenCV	fresh jobs	median s	min–max s	median end-to-end s	USD/h	USD per analysis (compute)
Graviton4 standard wheel	5.0.0	51	5.13	4.45–6.23	13.9	0.1595	0.00023
Graviton4 COOL	5.1.0-dev	48	5.09	3.99–6.35	13.0	0.1695	0.00024
local x86 laptop	5.0.0	51	3.74	3.03–4.79	—	—	—

Paired per-recording ratio COOL/standard over 17 recordings: median 1.003, geometric mean 0.996, range 0.87–1.04. A post-hoc bootstrap over the 17 recordings (10,000 resamples) puts the paired ratio at 1.003 with a 95% interval of 0.991–1.010, and the ratio of job-level medians at 0.991 (0.913–1.043) (`evidence/aws/speed-posthoc/result.json`): any difference is within about one per cent. **On this workload COOL gives no measurable speed-up.** The pipeline is dominated by Python control flow, PNG decoding and small-template matching rather than by the large-image kernels COOL optimises, so this is the expected and honest result; the value of COOL here is a verified Marketplace build of OpenCV 5 on Graviton4 that returns the same numbers as the reference engine. End-to-end latency (upload, queue, polling, result) is 13–14 s per recording either way; compute cost is about 0.02 cents per analysis at on-demand prices plus the $0.01/h Marketplace fee. Three of 51 COOL jobs on the long-running live worker ended `worker_lease_exhausted` (the client reports them as interrupted and does not retry); the fresh standard worker had none. Wake-up from a stopped worker to a finished job took 45.9 s (`evidence/aws/wake`).

Where the time goes (`evidence/aws/cool-kernels`, 23 October, post-hoc). Both engines were then timed on one c8g.xlarge instance, in three alternating rounds, on real frames and recordings. COOL is clearly faster on large-image kernels: connected components 4.7×, full-frame template matching 1.56× with four threads (1.0× with one), phase correlation 1.34×, bicubic upsampling 1.27× and corner detection 1.26×. It gains little on what this tracker mostly does: template matching in a 200 px search window (1.08×) and PNG decoding (1.01×). In process, the full measurement is 4.7% faster on a panel recording and 7.9% faster on an SVG recording; 54–65% of the profiled time is inside OpenCV, mostly `matchTemplate` and `imdecode`. In the service, the per-job overhead (S3 download, integrity checks, sandbox and process start) dilutes that gain to the null result above, which also compared two separate instances. Batching the tracker's matches into larger OpenCV calls is where COOL could help this pipeline. Both builds use the KleidiCV HAL; COOL is compiled with an SVE baseline, the standard wheel with NEON and runtime dispatch.

Scaling and cost per analysis (`evidence/aws/cool-scaling`, 23 October, post-hoc). The benchmark was extended and run, three rounds per engine, on c8g.xlarge and c8g.4xlarge (4 and 16 Graviton4 vCPUs, COOL and the standard wheel on each) and, with the standard wheel, on two x86 instances of the same size class: c8i.xlarge (Intel Xeon 6975P-C, two cores with two threads each) and c8a.xlarge (AMD EPYC 9R45, four cores). Three findings:

- *COOL's lead comes from threads and frame size.* The standard wheel runs full-frame template matching, corner detection and phase correlation on one thread whatever the setting; COOL parallelises them. With 16 threads, full-frame matching is 1.8× faster on the recording's 960 × 640 frames, 2.95× on 1080p and 3.2× on 4K frames tiled from the same recording. On one thread COOL is no faster (0.97–0.98× on matching), and the whole measurement is 2–3% slower.
- *For this tracker, one single-threaded job per vCPU is the efficient way to run.* It gives about 2.5 to 15 times the analyses per hour of one job at a time using every thread. In that mode COOL's parallel kernels do not apply and both builds deliver the same throughput (c8g.4xlarge: 14,126 against 14,269 analyses per hour, overlapping ranges). The live service runs one job at a time with four threads; there COOL's threads do help, measuring a recording 7% faster with four threads and 11% faster with sixteen.
- *Graviton4 costs about a third less per analysis than the Intel instance and about the same as the AMD one.* Compute cost with one job per vCPU, on-demand prices, service overhead excluded:

instance (one single-threaded job per vCPU)	analyses per hour, median of 3 rounds	USD per 1,000 analyses
c8g.4xlarge, Graviton4, standard wheel	14,269	0.045
c8g.4xlarge, Graviton4, COOL (fee included)	14,126	0.048
c8a.xlarge, AMD EPYC 9R45, standard wheel	5,016	0.043
c8i.xlarge, Intel Xeon 6975P-C, standard wheel	2,597	0.072

  On the 4-vCPU c8g.xlarge the rounds of both engines vary between about 2,900 and 3,800 analyses per hour, so the larger instance is the reliable Graviton4 figure (`REPORT.md` in the evidence folder lists every round).

5.3 Judge access and operations

The live endpoint has two judge accounts with permanent passwords, delivered privately with the submission; `submission/JUDGE_ACCESS.md` explains login and a one-command measurement. A dry run of that path from a clean clone of the public repository (fresh virtual environment, `pip install -e .`, both checks against the live COOL worker, `--compare-local` equivalent) passed on 23 October with an internal account (`evidence/aws/judge-dryrun`). The stack will be torn down after judging. Deployment used the account's root credentials and a hand-written IAM scope; a least-privilege deployer policy is listed under future work. No Lambda logging is enabled by design (no content leaves the tenant's objects), which we note as an observability trade-off.

6. Evaluation

6.1 Protocol

Every domain is evaluated the same way: calibration cases the agent sees, a known-regression set and a fresh validation set it never sees until sealing; pixel limits derived from a physical scale; independent review of calibration positions by AI reviewer agents who see only raw pixels; a source review of the candidate code; and a native audit of the agent's tool calls. A closure binds these axes to one source epoch and is validated by replaying it.

Who reviews. "Independent" in this report means independent of the production measurement, not human. Every raw-pixel, source and calibration review was performed by AI reviewer agents: separate Claude Sonnet sessions (and, in earlier phases, Codex) that receive only raw image sheets or source files and never the production coordinates, events or verdicts, followed by spot checks of disputed sheets by the orchestrating agent. The blind focus-ring marks were made the same way. The author set the protocols, approved runs and decisions and read the results, but did not hand-mark pixels, and no external human review is claimed. Raw positions used for the accuracy tables come from a separate extraction path in code that sees no production output.

6.2 Results

Domain and run	Calibration	Regression	Validation	Other axes	Verdict
Game, V4 (`v4g01`)	pass	48/48	30/30	raw review, source, native: pass	**accepted**
SVG, V5b (`v5s01c`, historical candidate re-measured, no new model generation)	4/4	12/12 known regression	24 held-out in the full matrix	112 raw records reviewed (244 sheets, 480 points); source axis pass; native origin historical	**accepted**
Panel, V4 (`v4p01`)	pass	27/36 (4 unobservable recipe cases, 5 pixel failures)	—	native fail	fail
Panel, V5 (`v5p01`)	21/22 for both candidates	—	not reached (no eligible candidate)	—	fail
Panel, V5b (`v5p02c`)	16/16, eligible	finalize aborted: capture stall	—	native pass	fail (infrastructure)
Panel, V6 demo (`v6p01`)	candidate 02 eligible, 0.0067	43/44	12/12	native uncertified (CLI 2.31)	fail

The V6 panel failure is a genuine regression caught by the held-out set: after a drag release at 1,440 px Kiro's panel springs back toward where the drag started, while the reference and the unchanged app travel on to the other side, so no settled rest is observed in the scored window (post-release position error 0.78–0.86 of the panel scale against limits of 0.05–0.076; 0.005–0.006 for the unchanged app). The calibration set contains no drag-release case, which is exactly what held-out cases are for.

Pass counts come from small samples. Exact two-sided 95% lower confidence bounds (Clopper–Pearson, computed after the fact) are 92.6% for 48/48, 88.4% for 30/30 and 89.1% for the 32/32 blind endpoint marks.

6.3 Recording reliability

Between 13 and 15 October seven long recordings (panel and SVG freezes and two finalizations) died on single host stalls of roughly 140–270 ms against a fixed 120 ms per-command deadline, including one that discarded 17 of 24 completed held-out cases. V6 added scenario-level re-recording after a timing stall (failed attempts are kept beside the canonical recording), a one-second host scheduling sample before each capture that refuses to start on a stalling machine, and an elevated priority class for the capture process and its Chromium children. In the V6 panel run all 224 final recordings passed their timing checks. No timing threshold was relaxed.

6.4 Tests

The official regression `full-tests-02` (`evidence/tests/full-tests-02.result.json`) ran 3,789 unit tests with no failures or errors and 39 skips, with source hashes verified unchanged before and after. A clean copy of the frozen tree without any evidence folders runs 3,755 tests green (173 marked skips name the missing evidence). The V6 source epoch is `C:/kc3d/code/v6b` (1,026 files, tree `fc724765…`).

Post-freeze fix (22 October). While preparing this report we found that `inspect_window` placed every window on the recording's first input and plotted absolute positions, whereas the comparison aligns each window on its own input and scores 68 of the 102 calibration windows of the V6 panel run relative to an anchor object. No recorded session ever called the tool (0 calls in 13 tool audits), so no result depends on it. The fix aligns and plots each window exactly as the comparison scored it. On those 102 windows the fixed inspection's error matches the comparison's own window RMSE with a median ratio of 0.999 (98 within 15%); the frozen V6 module gave a median ratio of 13.7 (`evidence/inspect-window-post-freeze`). The repository is frozen as source epoch `C:/kc3d/code/v6c` (1,038 files, tree `7962d423…`): `v6b` plus this fix, `README.en.md` and the AWS deployment, speed and judge scripts. Its full run (`full-tests-03`,

`evidence/tests/full-tests-03.result.json`) ran 3,791 tests: 3,748 passed, including the two new tests, and 40 were skipped. The three that failed re-verify proofs against Kiro CLI runtime folders (2.24.0 and 2.27.0) that the CLI's own update to 2.31.0 deleted on 16 October, after `full-tests-02`, in which the same three passed; one extra skip has the same cause. None of them touches the changed module. A clean copy of `v6c` without evidence folders, models or a Kiro installation, which is what a judge clones, runs 3,757 tests green with 173 skips that name the missing evidence or optional inputs (`clean-clone-02`). The public English edition of this tree, in which comments, messages and report text are translated and internal working documents are left out (`PUBLICATION.md`), runs the same 3,757 tests green with the same 173 skips (`evidence/tests/public-tree-tests.log`).

7. Limitations and responsible use

- Fixed-camera 2D renders with distinguishable objects; no hidden-state or input inference; visual proximity is not physical collision.
- A single uploaded clip cannot validate unseen behaviour; such results stay `partial`. Only the pre-registered multi-clip suites yield acceptance.
- The web panel domain is not accepted. The learned-feature locator fails on tiny, low-texture glyphs.
- The guard is a cooperative boundary; the agent's shell is otherwise unrestricted inside the session. Detection (hash checks, native audit) is the backstop, not prevention.
- Native-audit certification covers kiro-cli 2.27.x only.
- The COOL worker's sandbox can reach the instance metadata service; deployment used root credentials. Both are documented, neither is hidden.
- No external user study; "real-world benefit" is limited to the measured improvements.
- No human reviewer marked pixels or audited code; all independent reviews were done by separate AI agents under the protocols above.
- Licensing: the repository is MIT; third-party game assets and the COOL AMI keep their own licences (`config/third-party-notices.json`). No personal data is processed; recordings are synthetic renders of fixture applications.

8. Reproducibility

```
.\scripts\bootstrap_local.ps1                      # Python 3.10 venv, pinned wheels (requirements-
local.lock)
.\.venv-local\Scripts\python.exe -m kiroclops.local_engine doctor
.\.venv-local\Scripts\python.exe scripts\fetch_opencv5_models.py   # ALIKED + LightGlue, SHA-1 verified
.\.venv-local\Scripts\python.exe -m unittest discover -s tests     # 3,791 tests (clean clone: 3,757,
evidence tests skip)
.\.venv-local\Scripts\python.exe scripts\v6_equivalence.py --records-root <calibration> --output <new folder>
python -m services.analysis_api.deploy start --output <folder>     # standard engine
python scripts\phase4_aws_deploy.py start --engine cool --output <folder>
```

The core package installs on any OS with `pip install -e .` (OpenCV 5.0.0.93, NumPy 2.2.6); web capture adds Playwright 1.63.0 and Kiro sessions add `mcp` 2.2.0 (`pip install -e .[kiro]`). The public subset of the evidence is in `evidence/` with a SHA-256 manifest; Appendix A maps it to the internal sources.

9. Questions a judge may ask

Is Kiro just the assistant that wrote this entry? No. The rules exclude "using an AI coding assistant to write the entry"; here Kiro CLI is the agent under test at run time. KiroClops starts a Kiro session on a copy of the target app, Kiro calls the KiroClops tools, and OpenCV 5 measurements decide what it does next (§4.2). How the entry itself was built, with AI coding and review agents, is disclosed separately in §6.1.

Why does COOL not make the service faster? This tracker spends its time where COOL gains least: Python control flow, PNG decoding and 48 px templates matched in 200 px windows. COOL's lead grows with threads and frame size (full-frame matching 3.2× faster on 4K frames with 16 threads), and the in-process measurement gains 7–11% with four

to sixteen threads; per-job overhead absorbs the rest, and with one single-threaded job per vCPU, the cheapest way to run this tracker, both builds tie (§5.2). We report that as measured rather than pick a kernel that flatters COOL.

Why AI reviewers and not people? This is a solo entry. "Independent" means independent of the production measurement: reviewers see only raw pixels or source files, never production coordinates, events or verdicts (§6.1). No human pixel review is claimed; adding one would strengthen the accepted domains.

Why rendered recordings and not a camera? The targets are rendered software, so the rendered frame is the ground truth, and headless capture with per-frame clocks makes every candidate re-recordable to the frame, which sealed held-out cases need. The same OpenCV 5 core can track camera footage, but camera timing and lighting would need new, separately calibrated thresholds; we have not tested that.

Can I check it without Windows or a Kiro subscription? Yes, for the measurement: the core and the cloud client install on any OS, and `JUDGE_ACCESS.md` needs no Kiro. The agent loop needs Windows, Chromium capture and Kiro CLI; the recorded traces and the session replay show it.

How do I know the thresholds were not tuned afterwards? Recipes and limits were pre-registered and hash-bound before each run, held-out cases stay sealed until the agent finalizes, a rerun always gets a new identifier, and analyses done after a run are labelled post-hoc (§4.2 baseline, §5.2 kernel profile, §6.2 confidence bounds).

Appendix A. Evidence index

The public repository carries an English subset of the evidence in `evidence/`; `evidence/MANIFEST.json` records each file's internal source and both SHA-256 values. Items without a public copy (frames, full closure receipts, Kiro session workspaces and internal working logs) can be shown on request.

item	public copy	internal source
Game closure	`evidence/closures/game-v4g01-closure-02.summary.json`	`C:/kc3d/r/v4g01-closure-02` (summary, receipt, replay validation)
SVG closure	`evidence/closures/svg-v5s01c-closure-01.summary.json`, `evidence/svg-focus-ring-blind-qualification.json`	`C:/kc3d/r/v5s01c-closure-01`; raw proof `C:/kc3d/r/v5s01c-final-raw-proof-01.json`; `C:/kc3d/r/v5s-ring-score-01`
V6 panel run	`evidence/panel-v6p01/heldout-outcome.json`, `evidence/v6-agent-baseline/result.json`, `submission/evidence/agentic-trace-v6p01/`, `submission/AGENTIC_VISION.md`	session `C:/kc3d/s/v6p01`; results log `reviews/phase4-v6/v6-results.md`; `artifacts/phase4-v6/agent-baseline-posthoc-01`
Earlier panel runs (V4, V5, V5b)	—	`reviews/phase3d-v4/v4-results.md`, `reviews/phase3d-v5/v5-results.md`
All 13 logged agent sessions (§4.2)	`evidence/agent-sessions/`	sessions `C:/kc3d/s/*`; `artifacts/phase4-v6/agent-sessions-posthoc-01`
OpenCV 5 measurement (§3)	`evidence/{opencv5-symbol-diff, lightglue-sweep, lightglue-platforms, subpixel-benchmark, marks-error, feature-cross-check, v6-tracker-equivalence}`	`artifacts/phase4-v6/{opencv-symbol-diff-01, lightglue-sweep-01, lightglue-platforms-01, lightglue-5x-check-01..04, subpixel-benchmark-01, marks-error-01, cross-check-01, equivalence-01}`
AWS and COOL (§5)	`evidence/aws/{standard-equivalence, cool-equivalence, speed, speed-posthoc, cool-kernels, cool-scaling, wake, judge-dryrun}`	`artifacts/phase4-aws/{v6-equivalence-01, v6-cool-equivalence-02, speed-01, speed-01-posthoc-01, cool-kernels-01, cool-scaling-01, live-cool-wake-02, live-cool-02, judge-dryrun-01}`
Tests and the post-freeze fix (§6.4)	`evidence/tests/`, `evidence/inspect-window-post-freeze/result.json`	`artifacts/phase4-v6/{full-tests-02, full-tests-03, clean-clone-01, clean-clone-02, post-freeze-inspect-window-01}`

item	public copy	internal source
Source epochs	`evidence/source-epochs.json`	`C:/kc3d/code/{v4-final, v5b, v6, v6b, v6c}` with `epoch.json` manifests

Source: submission/TECHNICAL_REPORT.md in github.com/tritonsan/kiroclops. Paths in code font refer to that repository.